

Learning for Data Science

Course 8: Feature Engineering

Glen Berseth

Université de Montréal and Mila Québec AI Institute

September 23, 2026



Université
de Montréal



Mila

CIFAR

Outline

Standardization

Generation of balanced datasets

Image type data augmentation

Increase in tabular data

Data Drift: A Common Situation

Standardization

- Normalization (or min-max normalization) scales all values in a fixed range between 0 and 1.

$$X_{norm} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- This transformation does not change the distribution of the feature.
- But due to the decreasing standard deviations, the effects of outliers increase. So before normalization, it is recommended to deal with outliers.

Standardization

- Example: Min-Max scaling
- It uses the following formula for each value (X) in the dataset:
- $X_{\min}$ is the minimum value in the feature.
- $X_{\max}$ is the maximum value in the feature.

$$X_{scaled} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

```
data = pd.DataFrame({'value':[2,45, -23, 85, 28, 2, 35, -12]})  
  
data['normalized'] = (data['value'] - data['value'].min()) /  
(data['value'].max() - data['value'].min())
```

	value	normalized
0	2	0.23
1	45	0.63
2	-23	0.00
3	85	1.00
4	28	0.47
5	2	0.23
6	35	0.54
7	-12	0.10

Standardization

- Standardization (or z-score normalization) scales the values while accounting for the standard deviation.
- In the following standardization formula, the mean is represented by μ and the standard deviation is represented by σ .

$$z = \frac{x - \mu}{\sigma}$$

- If the standard deviation of the features is different, their range would also be different from each other. This reduces the effect of outliers in the features.

Standardization

- Example:

```
data = pd.DataFrame({'value':[2,45, -23, 85, 28, 2, 35, -12]})  
  
data['standardized'] = (data['value'] - data['value'].mean()) /  
data['value'].std()
```

	value	standardized
0	2	-0.52
1	45	0.70
2	-23	-1.23
3	85	1.84
4	28	0.22
5	2	-0.52
6	35	0.42
7	-12	-0.92

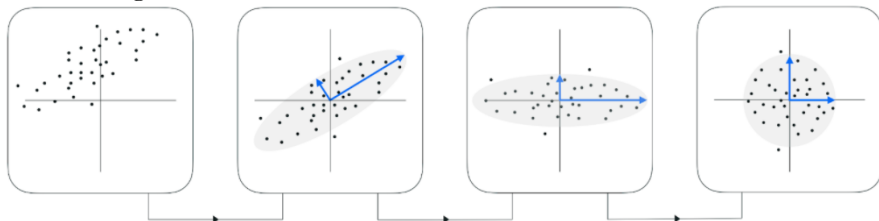
- Potential issues:
 - What if the std is small or large?
 - Clean data first

Data laundering

- Method based on principal component analysis (PCA).
- It centers the mean of the data x on the origin, removes the covariances and makes the variance unitary.

- Original data x
- Centering

- Whitened data z
- Normalization and Reverse Rotation



- Rotation of the main axes

→

Outliers

- Outliers can be introduced into the population during data collections
 - Which number is the potential outlier?

Players	Scores
Player1	500
Player2	350
Player3	10
Player4	300
Player5	450

- Mistake ?
- Difference ?

Treatment of outliers

- Common origins of outliers:

- Instrumental errors
- Natural noise in data
- Bad entries in an Excel file
- Mislabeled data
- New phenomenon!



Clipart Of.com
#439431

- Why are outliers bad?:

- Can cause large losses
- Many ML models have assumptions on data range

Detection of outliers

- Outlier: A data object that deviates significantly from normal objects as if generated by a different mechanism

Detection of outliers

- Outlier: A data object that deviates significantly from normal objects as if generated by a different mechanism
- Example: Unusual purchase by credit card, sports: Michael Jordon, Wayne Gretzky, ...
 - Outliers are different from noise data.
 - Noise is random error or variance in a measured variable
 - Noise should be removed before outlier detection
 - Outliers are interesting. It violates the mechanism that generates the normal data
 - Outlier detection versus novelty detection: early stage, outlier; but later merged into the model

Detection of outliers

- Outlier: A data object that deviates significantly from normal objects as if generated by a different mechanism
- Example: Unusual purchase by credit card, sports: Michael Jordon, Wayne Gretzky, ...
 - Outliers are different from noise data.
 - Noise is random error or variance in a measured variable
 - Noise should be removed before outlier detection
 - Outliers are interesting. It violates the mechanism that generates the normal data
 - Outlier detection versus novelty detection: early stage, outlier; but later merged into the model
- Applications:
 - Credit Card Fraud Detection
 - Telecom fraud detection
 - Customer segmentation
 - Medical analysis

Types of outliers

- 3 types: Global, contextual or grouped.
- **Global** outlier:
 - The value deviates significantly from the rest of the data.
 - Example: Intrusion detection in a computer network.
 - Problem: Finding a suitable deviation measure.
- **Contextual** outlier:
 - The value deviates significantly in a particular context
 - Example: 40 degrees Celsius in Montreal in winter.
 - Data attributes should be divided into two groups:
 - Contextual attributes, e.g., times, places,...
 - Behavioral attributes: measured values that may be outliers.

Types of outliers

- 3 types: Global, contextual or grouped.
- **Collective** outliers:
 - A subset of the data collectively deviates significantly from the data set, even though individual data objects may not be outliers.
 - Applications: For example, intrusion detection: When a number of computers keep sending each other denial of service packages
 - Detection of collective outliers
 - Consider not only the behavior of individual objects, but also that of groups of objects.
 - Need to have the basic knowledge about the relationship between data objects, such as a measure of distance or similarity on the objects.

Example of handling 2-D outliers



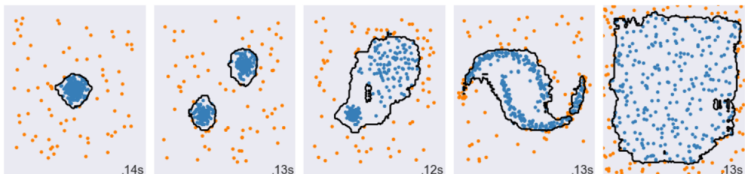
- Generation of ideal data



- Adding outliers

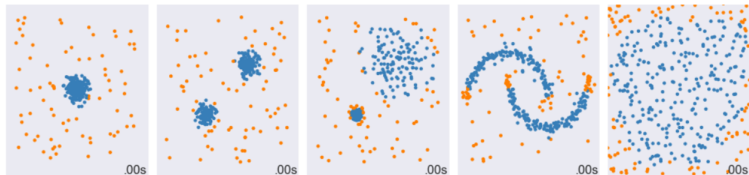
Example of handling 2-D outliers

Isolation Forest



- Remaining values after eliminating outliers.

Local Outlier Factor



Find outliers

- Visualization tools or statistical methodologies
 - Boxplot
 - A cloud of dots
 - Z-score
 - IQR score
- Advanced Anomaly Detection Techniques (Feature Selection Course)

Finding Outliers: Box plot

- Box plot

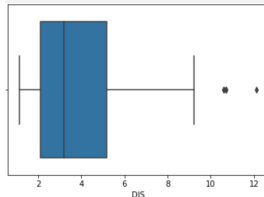
Finding Outliers: Box plot

- Box plot
- Left of the Box (Q1): This line marks the first quartile. 25% of the data points are below this value.

Finding Outliers: Box plot

- Box plot
- Left of the Box (Q1): This line marks the first quartile. 25% of the data points are below this value.
- Middle Line (Median or Q2): This is the median of the dataset. It splits the data in half—50% of the data points are below it, and 50% are above it.

```
import seaborn as sns
sns.boxplot(x=boston_df['DIS'])
```



Boxplot — Distance to Employment Center

Finding Outliers: Box plot

- Right of the Box (Q3): This line marks the third quartile. 75% of the data points are below this value.

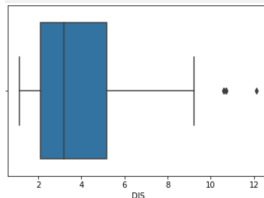
Finding Outliers: Box plot

- Right of the Box (Q3): This line marks the third quartile. 75% of the data points are below this value.
- Right/upper Whisker: Extends from the third quartile (Q3) to the largest data point that is within Q3.

Finding Outliers: Box plot

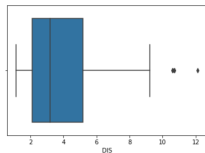
- Right of the Box (Q3): This line marks the third quartile. 75% of the data points are below this value.
- Right/upper Whisker: Extends from the third quartile (Q3) to the largest data point that is within Q3.
- Left/lower Whisker: Extends from the first quartile (Q1) to the smallest data point that is within from Q1.
- <https://www.jmp.com/en/statistics-knowledge-portal/exploratory-data-analysis/box-plot>

```
import seaborn as sns
sns.boxplot(x=boston_df['DIS'])
```



Boxplot — Distance to Employment Center

Finding Outliers: Box plot

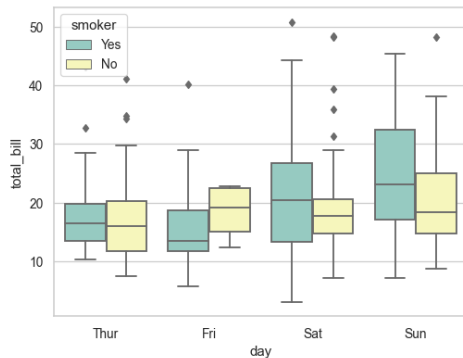


Boxplot — Distance to Employment Center

- Three points between 10 to 12 -> outliers as there are not included in the box of other observation i.e no where near the quartiles.
- Here: Uni-variate outlier, i.e. only the DIS column is used to check for outliers.
- Multivariate outlier analysis is also possible. Can a box plot support multivariate analysis? It depends: a categorical variable can be paired with any continuous variable for multivariate outlier analysis.

Outliers: boxplots

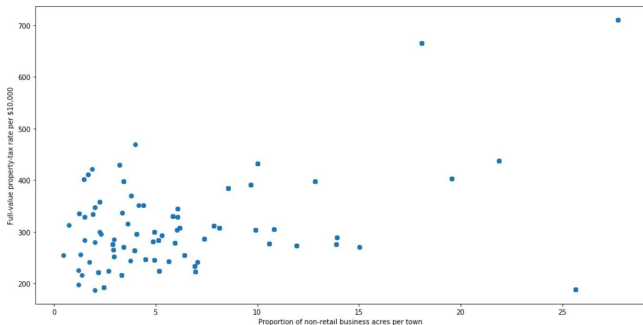
- Boxplot with nested grouping by two categorical variables
- `ax = sns.boxplot(x="day", y="total_bill", hue="smoker", ... data=tips, palette="Set3")`



Finding Outliers: Scatter plot

- Scatter plot for two variables from the housing dataset.

```
fig, ax = plt.subplots(figsize=(16,8))
ax.scatter(boston_df['INDUS'], boston_df['TAX'])
ax.set_xlabel('Proportion of non-retail business acres per town')
ax.set_ylabel('Full-value property-tax rate per $10,000')
plt.show()
```



Scatter plot — Proportion of non-retail business acres per town v/s Full value property tax

Finding Outliers: STD

- Metric to identify outliers: standard deviation
- If a value has a distance to the average higher than x * standard deviation, it can be assumed as an outlier. Then what x should be?
- Usually, a value between 2 and 4 seems practical.

$$\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{N}}$$

σ = population standard deviation, N = the size of the population, x_i = each value from the population, μ = the population mean

```
#Dropping the outlier rows with standard deviation
factor = 3
upper_lim = data['column'].mean () + data['column'].std () * factor
lower_lim = data['column'].mean () - data['column'].std () * factor

data = data[(data['column'] < upper_lim) & (data['column'] >
lower_lim)]
```

Finding Outliers

- Z-score
- The Z-score is the signed number of standard deviations by which the value of an observation or data point is above the mean value of what is being observed or measured.
- Wikipedia Definition
- The intuition behind Z-score is to describe any data point by finding their relationship with the Standard Deviation and Mean of the group of data points. Z-score is finding the distribution of data where mean is 0 and standard deviation is 1 i.e. normal distribution.

Finding Outliers: Z-score

- Wikipedia definition:
 - The Z-score is the signed number of standard deviations by which the value of an observation or data point is above the mean value of what is being observed or measured.
- Re-scale and center the data
- Outliers: data points which are too far from zero.
- In most of the cases a threshold of 3 or -3 is used

```
from scipy import stats
import numpy as np

z = np.abs(stats.zscore(boston_df))
print(z)
```

```
[[0.41771335 0.28482986 1.2879095 ... 1.45900038 0.44105193 1.0755623 ]
 [0.41526932 0.48772236 0.59338101 ... 0.30309415 0.44105193 0.49243937]
 [0.41527165 0.48772236 0.59338101 ... 0.30309415 0.39642699 1.2087274 ]
 ...
 [0.41137448 0.48772236 0.11573841 ... 1.17646583 0.44105193 0.98304761]
 [0.40568883 0.48772236 0.11573841 ... 1.17646583 0.4032249 0.86530163]
 [0.41292893 0.48772236 0.11573841 ... 1.17646583 0.44105193 0.66905833]]
```

Z-score of Boston Housing Data

Finding Outliers

- From the code and the output above, it is difficult to say which data point is an outlier. The next step is to define a threshold that identifies outliers.

```
threshold = 3  
print(np.where(z > 3))
```

```
(array([ 55,  56,  57, 102, 141, 142, 152, 154, 155, 160, 162, 163, 199,  
       200, 201, 202, 203, 204, 208, 209, 210, 211, 212, 216, 218, 219,  
       220, 221, 222, 225, 234, 236, 256, 257, 262, 269, 273, 274, 276,  
       277, 282, 283, 284, 347, 351, 352, 353, 353, 354, 355, 356,  
       357, 358, 363, 364, 364, 365, 367, 369, 370, 372, 373, 374, 374,  
       380, 398, 404, 405, 406, 410, 410, 411, 412, 412, 414, 414, 415,  
       416, 418, 418, 419, 423, 424, 425, 426, 427, 427, 429, 431, 436,  
       437, 438, 445, 450, 454, 455, 456, 457, 466], dtype=int64), array([ 1,  1,  1, 11, 12,  3,  3,  
  3,  3,  3,  3,  1,  1,  1,  1,  1,  
  1,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  5,  3,  3,  1,  5,  
  5,  3,  3,  3,  3,  3,  1,  3,  1,  1,  7,  7,  1,  7,  7,  7,  
  3,  3,  3,  3,  3,  5,  5,  5,  3,  3,  3, 12,  5, 12,  0,  0,  0,  
  0,  5,  0, 11, 11, 11, 12,  0, 12, 11, 11,  0, 11, 11, 11, 11, 11,  
 11,  0, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11]),  
dtype=int64))
```

List of row
indices

List of
column
indices

Data points where Z-scores is greater than 3

Finding Outliers

- The interquartile range (IQR):
 - Also called the midspread or middle 50%, or technically H-spread, is a measure of statistical dispersion, being equal to the difference between 75th and 25th percentiles, or between upper and lower quartiles;
 - $IQR = Q_3 - Q_1$.
- Remark: It is a measure of the dispersion similar to standard deviation or variance but is much more robust against outliers.

Finding Outliers

- Box plots use the IQR, which can also produce the list of outliers, as the Z-score calculation did. The first step is to calculate the IQR.

```
Q1 = boston_df_o1.quantile(0.25)
Q3 = boston_df_o1.quantile(0.75)
IQR = Q3 - Q1
print(IQR)
```

```
CRIM      3.565378
ZN       12.500000
INDUS    12.910000
CHAS      0.000000
NOX      0.175000
RM       0.738000
AGE      49.050000
DIS       3.088250
RAD      20.000000
TAX     387.000000
PTRATIO   2.800000
B        20.847500
LSTAT    10.005000
dtype: float64
```

IQR for each column

Finding Outliers

- A value of False means the data point is valid, whereas True indicates the presence of an outlier.

```
print(boston_df_o1 < (Q1 - 1.5 * IQR)) |(boston_df_o1 > (Q3 + 1.5 * IQR))
```

4	False	False	False	False	False	False	False	False	False	False	False	False	False
5	False	False	False	False	False	False	False	False	False	False	False	False	False
6	False	False	False	False	False	False	False	False	False	False	False	False	False
7	False	False	False	False	False	False	False	False	False	False	False	False	False
8	False	False	False	False	False	False	False	False	False	False	False	False	False
9	False	False	False	False	False	False	False	False	False	False	False	False	False
10	False	False	False	False	False	False	False	False	False	False	False	False	False
11	False	False	False	False	False	False	False	False	False	False	False	False	False
12	False	False	False	False	False	False	False	False	False	False	False	False	False
13	False	False	False	False	False	False	False	False	False	False	False	False	False
14	False	False	False	False	False	False	False	False	False	False	False	False	False
15	False	False	False	False	False	False	False	False	False	False	False	False	False
16	False	False	False	False	False	False	False	False	False	False	False	False	False
17	False	False	False	False	False	False	False	False	False	False	False	False	False
18	False	False	False	False	False	False	False	False	False	False	True	False	False
19	False	False	False	False	False	False	False	False	False	False	False	False	False
20	False	False	False	False	False	False	False	False	False	False	False	False	False
21	False	False	False	False	False	False	False	False	False	False	False	False	False

Detecting outlier with IQR

Finding Outliers: Percentile

- Another mathematical method to detect outliers is to use percentiles.
 - A percentile (or a centile) is a measure used in statistics indicating the value below which a given percentage of observations in a group of observations falls.
- You can assume a certain percent of the value from the top or the bottom as an outlier.
- A common mistake is using the percentiles according to the range of the data, e.g., if your data ranges from 0 to 100, your top 5% is not the values between 96 and 100. Top 5% means the values that are out of the 95th percentile of data.

Handling Outliers

- An Outlier Dilemma: Drop or Cap
- Correcting

```
#Capping the outlier rows with Percentiles
upper_lim = data['column'].quantile(.95)
lower_lim = data['column'].quantile(.05)
```

```
data.loc[(df[column] > upper_lim),column] = upper_lim
data.loc[(df[column] < lower_lim),column] = lower_lim
```

- Removing

```
boston_df_o = boston_df_o[(z < 3).all(axis=1)]
```

- Z-score:
- IQR score:

```
boston_df_out = boston_df_o1[~((boston_df_o1 < (Q1 - 1.5 * IQR)) |
(boston_df_o1 > (Q3 + 1.5 * IQR))).any(axis=1)]
```

```
boston_df_out.shape
```

Grouping

- Tidy data where each row represents an instance and each column represent a feature.

Tidy datasets are easy to manipulate, model and visualise, and have a specific structure: each variable is a column, each observation is a row, and each type of observational unit is a table.

— Hadley Wickham

- Transaction datasets rarely fit the definition of tidy data -> grouping is used.
- The key point of group by operations is to decide the aggregation functions of the features.

Grouping

- Aggregating categorical columns:
- Highest frequency: the max operation for categorical columns

```
data.groupby('id').agg(lambda x: x.value_counts().index[0])
```

- Make a Pivot table: This would be a good option if you aim to go beyond binary flag columns and merge multiple features into aggregated features, which are more informative.

User	City	Visit Days
1	Roma	1
2	Madrid	2
1	Madrid	1
3	Istanbul	1
2	Istanbul	4
1	Istanbul	3
1	Roma	3



User	Istanbul	Madrid	Roma
1		3	4
2		4	0
3	1	0	0

Pivot table example: Sum of Visit Days grouped by Users

- Apply one-hot encoding

Grouping

- Numerical columns are mostly grouped using:
- Sum
- Mean

```
#sum_cols: List of columns to sum
#mean_cols: List of columns to average

grouped = data.groupby('column_to_group')

sums = grouped[sum_cols].sum().add_suffix('_sum')
avgs = grouped[mean_cols].mean().add_suffix('_avg')

new_df = pd.concat([sums, avgs], axis=1)
```

More feature engineering...

- Digital data
 - Dimensionality Reduction Techniques - SVD and PCA (Unsupervised Learning Week 7)
 - Grouping and use of cluster IDs or/and distances from cluster centers as new features
 - Unsupervised learning
 - Feature Selection
- Text data
 - For example, Bag-of-Words representations
- Image data:
 - Filters
 - Convolutions
- Relational data (graphs):
 - Representations of nodes.

Generation of balanced datasets



Tactics to Combat Class Imbalance

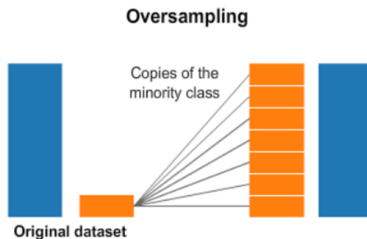
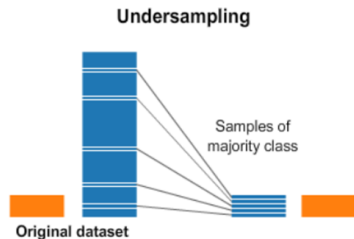
- Here are the main ones:

Fix the data first

- Collect more data from minority classes!!!
 - Resample the dataset.
 - Generate synthetic samples (data augmentation).
-
- Change performance metric.
 - Use a loss function taking into account the probabilities of the classes.

Resampling a dataset

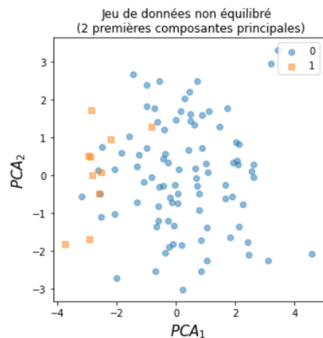
- When to oversample or undersample?



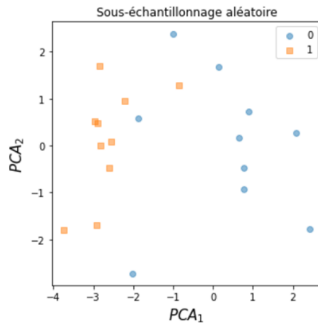
- With a lot of data

- With little data

Simple downsampling example

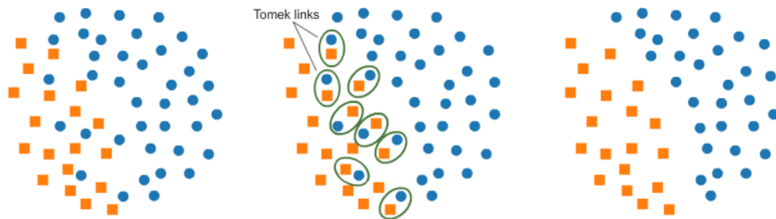


- Original dataset
- 90% class 0 in blue
- 10% class 1 in orange



- The majority class was undersampled.

Example of subsampling Tomek links



- Tomek links eliminate majority-class points that are close to minority-class points. The class boundary becomes better defined.

Examples of Data Imbalance Cases

- Tabular data:
 - Finance: fraudulent transactions
 - Marketing: link clicked in an advertising web page
 - Medical: cancer detection
 - Email: spam
- Image data:
 - Traffic: Lamborghini Aventador on the highway
 - Industry: faulty integrated circuit
 - Security: child lost in the crowd of a shopping center

Benefits of Data Augmentation

- Improves the performance of models
 - Add training data
 - Reduces class imbalance
 - Reduces model overfitting
 - Improves the generalization of models
- Reduces data collection and labeling costs
 - Databases (DB) are expensive to build
 - Companies don't share their comics
 - The models fly away, but the comics remain



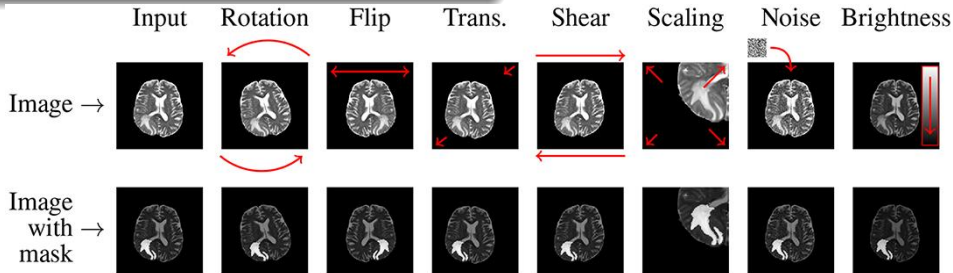
Image type data augmentation

Geometric changes

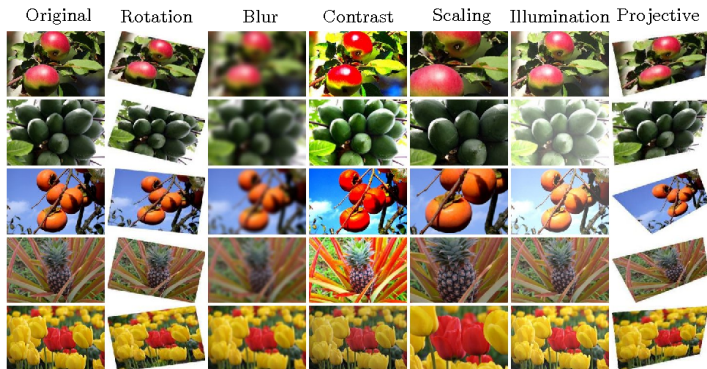
- Translation
- Random rotation
- Vertical and horizontal flipping
- Cropping
- Zoom

Intensity changes

- Change of colors
- Contrast change
- Adding noise
- Random erasing of areas in images



Examples of classic methods (2)



Methods based on neural networks

- Generative Adversarial Networks (GAN) (2014) (Goodfellow *et al.*, 2020)
- Neural Style Transfer (NST) (2015) (Gatys *et al.*, 2016)
- Text to image augmentation using diffusion (2023) (Trabucco *et al.*, 2024)

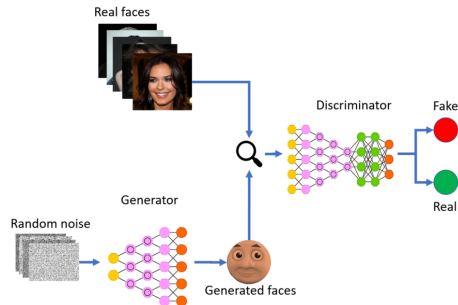


photo → Monet

- [Zhu et al. 2017]

Generative adversarial networks (GANs)

- A neural network learns to create new examples that resemble the training data.
- The Network Generator can then be used to create new realistic images!



Examples of synthetic images generated with GANs (1)

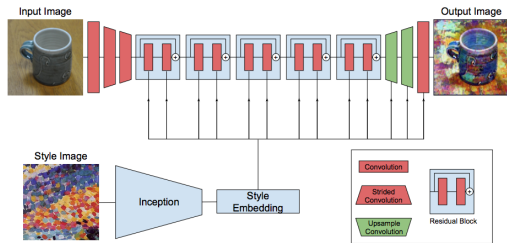


Examples of synthetic images generated with GANs



Neural Style Transfer

- A neural network extracts style elements from a reference image and applies them to the image to be modified.
- This allows multiple versions of the starting image to be generated using different style images.

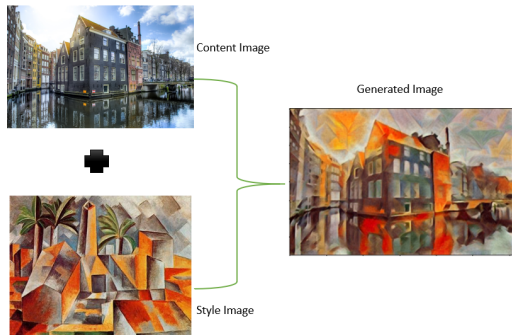


Examples of synthetic images generated by neural style transfer (1)

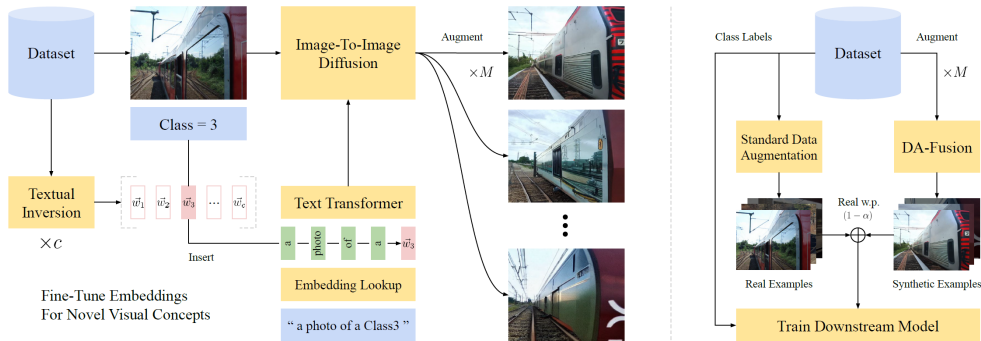


- Image
- Original

Examples of synthetic images generated by neural style transfer (2)



Examples of synthetic images generated by diffusion conditioned on text.



- Trabucco et al. 2023 Effective Data Augmentation With Diffusion Models

Increase in tabular data

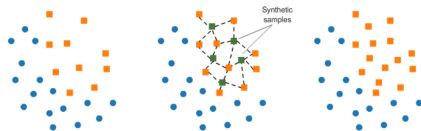
- Each entry in a table is a point in a D-dimensional space
- It is necessary to generate new points while respecting the boundaries in D dimensions of the table.
- Do not generate an age of -5.

	age (n)	job (c)	marital (c)	education (c)	balance (n)	housing (c)
0	30	unemployed	married	primary	1787	no
1	33	services	married	secondary	4789	yes
2	35	management	single	tertiary	1350	yes
3	30	management	married	tertiary	1476	yes
4	59	blue-collar	married	secondary	0	yes
5	35	management	single	tertiary	747	no

- Starters
- 6-D space

Classic methods

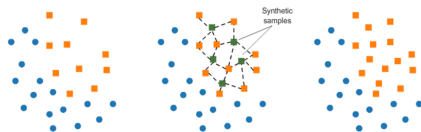
- SMOTE generates new elements of the disadvantaged class by generating points in the neighborhood of existing points.



- ADASYN randomly modifies the positions of the new SMOTE points.

Classic methods

- SMOTE generates new elements of the disadvantaged class by generating points in the neighborhood of existing points.



- ADASYN randomly modifies the positions of the new SMOTE points.
- Methods based on neural networks
 - Tabular GAN uses GANs to generate synthetic values that respect constraints on table value ranges.
 - TabularLLM uses a neural network to generate synthetic values that respect constraints on table value ranges()

Golden Rule



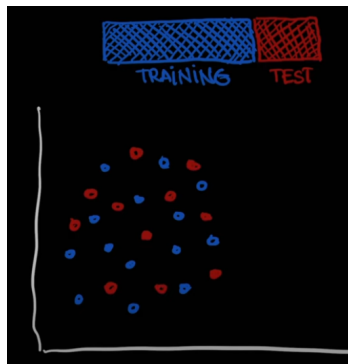
- Nothing beats adding real data!

Data Drift: A Common Situation

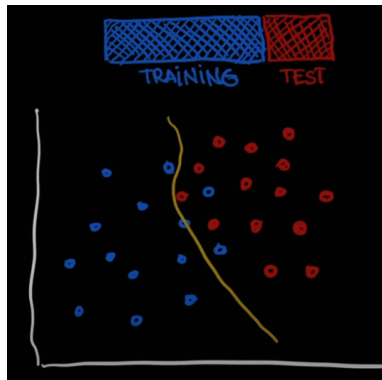
- A company has collected data in the last 6 months
- What you decide to do:
 - Take the first 5 months for training and validation
 - Take last month for test data
- The model performs very well on the training and validation sets
- The model does not perform well on the test set
- Q: What's going on????
- Answer: The test set is not from the same distribution.
- Inspired by: <https://www.youtube.com/watch?v=c5ClgEhAhK0&t=1s>

Antagonist validation, illustration

- No drift¹



- Drift



¹Source: <https://www.youtube.com/watch?v=c5ClgEhAhK0>

Antagonist validation

- In red: test set
- In blue: training set
- Idea: Label 1 for training and 0 for test.
- If a classifier is able to differentiate the two sets $\Rightarrow$ the two distributions are different $\Rightarrow$ data drift.
- Otherwise no data drift
 - Time $\Rightarrow$

- Features ~~~~~ target

C1	C2	C3	C4	target	AYT
•	•	•	•	•	1
•	•	•	•	•	1
•	•	•	•	•	1
•	•	•	•	•	1
•	•	•	•	•	1
•	•	•	•	•	0
•	•	•	•	•	0
•	•	•	•	•	0
•	•	•	•	•	0
•	•	•	•	•	0

Antagonist validation

[illegible]

- Targets: 0 1 0 1 0 1 0 101

[illegible]

- Target: 1011100011 ...

- The results must be compared to a random mixture (null hypothesis)
- The classifier should be worse at differentiating a random mixture.
- More statistical tests next week.

Data drift, what to do?

- If it's a constant drift: (hard)
 - Re-train the model on recent data constantly.
 - If not enough data: train and test on mixed data.
- If it is a drift that occurred at a specific time
 - Example: COVID.
 - Identify date (with previous test)
 - Train only after the change.

Conclusion

- This lecture covered:
 - Nomalisation
 - Treatment of outliers.
 - Data Balancing
 - Data increase
 - Data drift.

Refs used to build this conference

- <https://towardsdatascience.com/feature-engineering-for-machine-learning-3a5e293a5114>
- <https://towardsdatascience.com/ways-to-detect-and-remove-the-outliers-404d16608dba>
- <https://www.slideshare.net/0xdata/feature-engineering-83511751>
- https://ift6758.github.io/lectures/feature_engineering.pdf
- <https://towardsdatascience.com/target-encoding-and-bayesian-target-encoding-5c6a6c58ae8c>

Leon Gatys, Alexander Ecker, and Matthias Bethge. A neural algorithm of artistic style. *J. Vis.*, 16(12):326, September 2016.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. volume 63, page 139–144, New York, NY, USA, October 2020. Association for Computing Machinery.

Brandon Trabucco, Kyle Doherty, Max Gurinas, and Ruslan Salakhutdinov. Effective data augmentation with diffusion models. 2024:14590–14612, 2024.